

Introdução a Algoritmos

Aula 13a – Modelo de Memória e Alocação Dinâmica

Raoni F. S. Teixeira · 2s/2026

Objetivos desta aula

Ao final desta aula, você deverá ser capaz de:

1. Descrever os quatro segmentos de memória de um processo (código, dados, stack, heap) e classificar qualquer variável no segmento correto conforme sua declaração;
2. Explicar o mecanismo de frames da stack e por que variáveis locais deixam de existir após o retorno da função que as criou;
3. Alocar e liberar memória no heap com malloc, calloc e free, verificando o retorno de malloc antes de usar o ponteiro;
4. Identificar os três erros fatais com memória dinâmica: uso após free, double-free e vazamento de memória.

1 Pontas soltas

Ao longo do curso, deixamos algumas afirmações sem explicação completa:

- Na Aula 8: “variáveis locais são destruídas quando a função termina”. Por quê? O que significa “destruídas”?
- Na Aula 10: “char *s = "texto" aponta para memória somente-leitura”. Que memória é essa? Por que é somente-leitura?
- Novamente, na Aula 8: “nunca retorne o endereço de uma variável local”. Por que exatamente isso é perigoso?

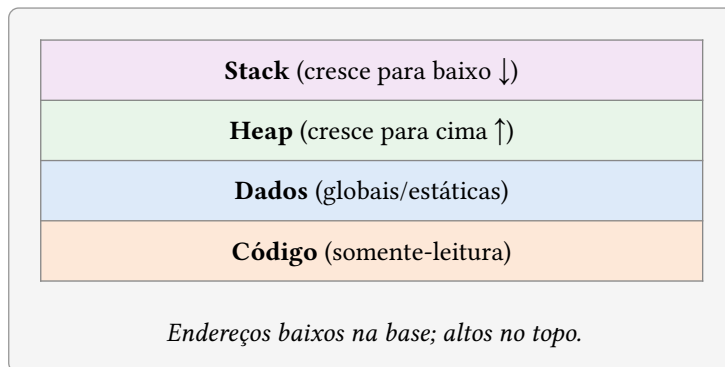
Todas essas respostas dependem do mesmo modelo: como a memória de um processo é organizada. Esta aula constrói esse modelo do zero.

2 Os segmentos de um processo

Quando o sistema operacional carrega um programa para execução, ele organiza a memória do processo em regiões com propósitos distintos. Em sistemas Unix (Linux, macOS) e Windows, a organização clássica tem quatro segmentos principais:

Segmento	O que contém	Características
Código (text)	Instruções do programa compilado; literais de string	Somente-leitura; tamanho fixo em tempo de compilação
Dados (data/bss)	Variáveis globais e estáticas inicializadas e não-inicializadas	Leitura e escrita; tamanho fixo em tempo de compilação
Stack (pilha)	Variáveis locais, parâmetros de função, endereços de retorno	Gerenciado automaticamente; cresce e encolhe com chamadas de função
Heap	Memória alocada dinamicamente pelo programa	Gerenciado pelo programador; tamanho determinado em tempo de execução

Visualizando o espaço de endereços de um processo (endereços crescem para cima):



3 O segmento de código e os literais de string

O segmento de código contém as instruções geradas pelo compilador — os bytes que o processador executa. Junto com as instruções, o compilador armazena nesse segmento os **literais de string** que aparecem no código-fonte.

Quando você escreve:

```
char *msg = "Erro";
```

O compilador coloca a sequência de bytes ‘E’ ‘r’ ‘r’ ‘o’ no segmento de código, e `msg` recebe o endereço desse local. O sistema operacional mapeia o segmento de código como **somente-leitura** — por razões de segurança (impede que o programa sobrescreva suas próprias instruções) e eficiência (múltiplos processos podem compartilhar o mesmo segmento).

Ponta resolvida

A ponta solta da Aula 10: agora sabemos por que `char *s = "texto"` aponta para memória somente-leitura — o literal `"texto"` vive no segmento de código, que o sistema operacional protege contra escrita. Tentar `s[0] = 'T'` tenta escrever nesse segmento protegido — o que causa uma violação de acesso imediata.

`char buf[] = "texto"` é diferente: o compilador **copia** os bytes do segmento de código para um buffer na stack local, e esse buffer é modificável normalmente.

4 A stack: memória automática

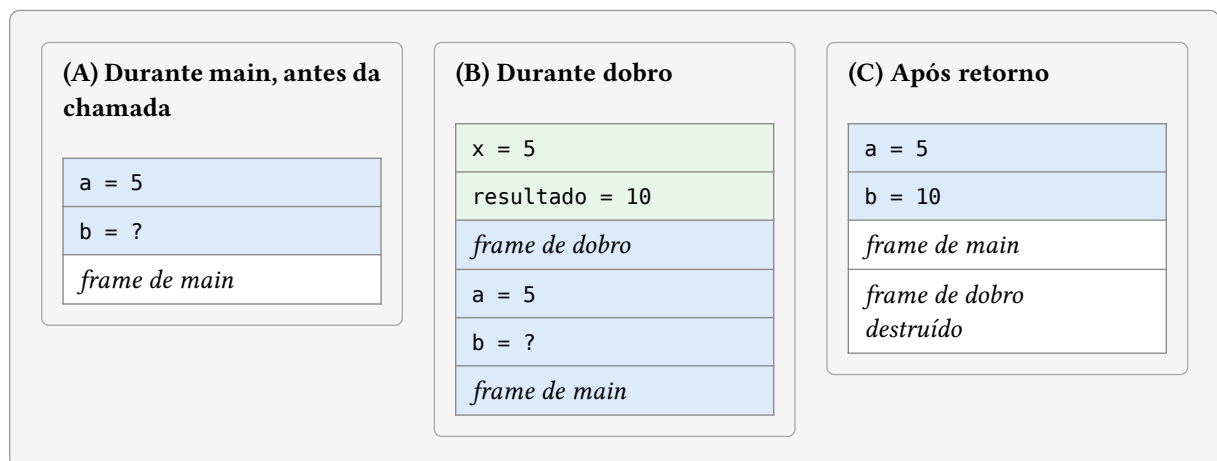
A stack (pilha) é a região onde vivem as variáveis locais e os parâmetros de função. Ela é gerenciada automaticamente pelo compilador, seguindo um mecanismo simples: cada chamada de função empilha um **frame** (quadro) contendo suas variáveis locais; quando a função retorna, o frame é desempilhado.

4.1 Como a stack funciona

Considere o seguinte código:

```
int dobro(int x) {  
    int resultado = x * 2;    /* (B) frame de dobro: x, resultado */  
    return resultado;  
}  
  
int main() {  
    int a = 5;                /* (A) frame de main: a, b */  
    int b = dobro(a);         /* chama dobro */  
    printf("%d\n", b);        /* (C) volta para main */  
    return 0;  
}
```

O estado da stack em cada momento:



Quando `dobro` retorna, seu frame é simplesmente descartado — o ponteiro da stack volta para a posição anterior. As variáveis `x` e `resultado` continuam nos mesmos bytes de memória por um instante, mas aquela região está agora disponível para ser reutilizada pela próxima chamada de função.

Ponta resolvida

A **ponta solta da Aula 8**: “nunca retorne o endereço de uma variável local” — porque o frame da função é destruído no retorno. O endereço aponta para bytes que serão sobrescritos pela próxima chamada.

```
int *perigo() {
    int x = 42;
    return &x;    /* x vive no frame de perigo – sera destruido */
}
int *p = perigo();
printf("%d\n", *p);    /* comportamento indefinido: frame ja reutilizado */
```

4.2 Limites da stack

A stack tem tamanho fixo (tipicamente 1–8 MB em sistemas modernos). Declarar arrays grandes como variáveis locais pode esgotar a stack:

```
void f() {
    int matriz[10000][10000];    /* 400 MB na stack – stack overflow! */
}
```

Para dados grandes, a solução é o heap.

5 O heap: memória dinâmica

O heap é a região de memória que o programa controla explicitamente: solicita blocos quando precisa e os libera quando termina. Isso resolve dois problemas que a stack não resolve:

1. **Tamanho desconhecido em tempo de compilação** — “quantos alunos?” só é sabido em tempo de execução.
2. **Tempo de vida independente de funções** — dados que precisam sobreviver além do retorno da função que os criou.

5.1 malloc e calloc

A função malloc (memory allocate) reserva um bloco de bytes no heap e retorna um ponteiro para ele:

```
#include <stdlib.h>
```

```
int *v = malloc(n * sizeof(int));    /* aloca espaco para n inteiros */
```

malloc retorna void * — um ponteiro genérico que C converte automaticamente para qualquer tipo de ponteiro. Se a alocação falhar (memória insuficiente), retorna NULL.

calloc faz o mesmo, mas **inicializa todos os bytes com zero**:

```
int *v = calloc(n, sizeof(int));    /* aloca e zera */
```

A diferença: malloc(n * sizeof(int)) e calloc(n, sizeof(int)) alocam o mesmo espaço, mas o segundo garante que todos os elementos começam em 0.

Verificar o retorno é obrigatório:

```
int *v = malloc(n * sizeof(int));
if (v == NULL) {
    fprintf(stderr, "Erro: memoria insuficiente\n");
    return 1;
}
```

5.2 free: devolver a memória

Toda memória alocada com malloc ou calloc deve ser liberada com free quando não for mais necessária:

```
free(v);  
v = NULL; /* boa pratica: evita usar o ponteiro apos liberar */
```

free informa ao sistema operacional que aquele bloco está disponível para reutilização. Não liberar memória causa **vazamento de memória** (memory leak): o processo acumula blocos inutilizados até esgotar a memória disponível.

Atenção

Três erros fatais com o heap:

1. **Usar memória após liberar** (use-after-free): free(v); v[0] = 1; — comportamento indefinido, frequentemente causa crash ou corrupção.
2. **Liberar duas vezes** (double-free): free(v); free(v); — corrompe as estruturas internas do alocador.
3. **Não liberar** (memory leak): em programas longos, acumula memória até o sistema matar o processo.

Por isso, atribuir NULL ao ponteiro após free é boa prática: um segundo free(NULL) é inofensivo, e NULL é fácil de detectar.

6 Usando malloc na prática

6.1 Vetor de tamanho dinâmico

```
#include <stdio.h>  
#include <stdlib.h>  
  
int main() {  
    int n;  
    printf("Quantos elementos? ");  
    scanf("%d", &n);  
  
    /* aloca n inteiros no heap */  
    int *v = malloc(n * sizeof(int));  
    if (v == NULL) { fprintf(stderr, "Sem memoria\n"); return 1; }  
  
    for (int i = 0; i < n; i++)  
        scanf("%d", &v[i]); /* v[i] funciona igual a um array */  
  
    int soma = 0;  
    for (int i = 0; i < n; i++)  
        soma += v[i];  
  
    printf("Soma: %d\n", soma);  
  
    free(v); /* devolve a memoria ao sistema */  
    v = NULL;
```

```
    return 0;
}
```

`v[i]` funciona exatamente como em um array estático — porque `v` é um ponteiro para o primeiro elemento, e `v[i]` é `*(v + i)`, como vimos na Aula 9. A única diferença é onde a memória vive (heap vs. stack) e quem gerencia seu tempo de vida (programador vs. compilador).

6.2 Struct alocada dinamicamente

```
typedef struct { char nome[100]; float media; } Aluno;
```

```
Aluno *cria_aluno(char *nome, float media) {
    Aluno *a = malloc(sizeof(Aluno));
    if (a == NULL) return NULL;
    strncpy(a->nome, nome, 99);
    a->nome[99] = '\0';
    a->media = media;
    return a;    /* seguro: a memoria vive no heap, nao no frame desta funcao */
}
```

```
int main() {
    Aluno *a = cria_aluno("Maria", 8.5);
    if (a != NULL) {
        printf("%s: %.1f\n", a->nome, a->media);
        free(a);
    }
    return 0;
}
```

`cria_aluno` retorna um ponteiro para o heap — seguro, porque a memória no heap sobrevive ao retorno da função. Contraste com retornar `&local`: a variável local estaria na stack e seria destruída.

7 Resumo: onde vive cada dado

Declaração	Segmento	Tempo de vida	Gerenciado por
<code>int x = 5;</code> (dentro de função)	Stack	Duração da função	Compilador
<code>int x = 5;</code> (fora de função)	Dados	Duração do programa	Compilador
<code>static int x = 5;</code>	Dados	Duração do programa	Compilador
<code>char *s = "texto";</code>	Código	Duração do programa	Compilador (somente-leitura)
<code>malloc(n * sizeof(int))</code>	Heap	Até <code>free()</code> ser chamado	Programador

8 Exercícios

1. Explique o que está errado em cada trecho e o que pode acontecer em tempo de execução:

a)

```
int *aloca_vetor(int n) {  
    int v[100];  
    return v;  
}
```

b)

```
int *v = malloc(10 * sizeof(int));  
free(v);  
v[0] = 42;
```

c)

```
char *s = "0la";  
s[0] = 'o';
```

2. Reescreva a função do item (a) corretamente, usando malloc. Quem é responsável por chamar free no ponteiro retornado?
3. Escreva uma função float *le_vetor(int n) que aloca um vetor de n floats no heap, lê n valores do teclado e retorna o ponteiro. Escreva main de forma que: (a) chame le_vetor; (b) calcule e imprima a média; (c) libere a memória corretamente.
4. (Desafio) Escreva uma função Aluno *le_turma(int *n) que: (a) leia *n do teclado; (b) aloque um vetor de *n structs Aluno no heap; (c) preencha cada struct; (d) retorne o ponteiro. Em main, use o vetor, depois libere. Por que o parâmetro n é int * e não int?